

CLAUDIA MARIA LOPEZ BOMBIN

LUCÍA LOZANO ISAC

OCTUBRE, 2025

PRÁCTICA 2

**O SEA, ¿CÓMO QUE MI CLAVE PUBLICA ES COMO MI INSTA:
TODOS LA VEN, PERO NADIE LA ENTIENDE?**

MATEMÁTICA DISCRETA

DAVID ALFAYA SÁNCHEZ

GRUPO GB06

ÍNDICE

INTRODUCCIÓN	3
DISEÑO E IMPLEMENTACIÓN.....	5
MANUAL DE USUARIO	25
CONCLUSIONES	31
AUTORÍA.....	35
BIBLIOGRAFÍA.....	36

INTRODUCCIÓN

Este documento, realizado por Claudia Maria Lopez Bombin y Lucía Lozano Isac – alumnas de 2B del Grado en Ingeniería Matemática e Inteligencia Artificial en la Universidad Pontificia Comillas-, corresponde a la memoria complementaria a la segunda practica de la asignatura Matemática Discreta, encuadrada en las nociones de cifrado y criptografía modular estudiadas.

Los fundamentos del cifrado modular que hoy protegen nuestras comunicaciones digitales encuentran sus raíces en la aritmética modular desarrollada por Carl Friedrich Gauss en su obra *Disquisitiones Arithmeticae* de 1801. Esta disciplina matemática, que Gauss definió como “el estudio de las congruencia entre números enteros” (Gauss, 1801), se ha revelado como la piedra angular de sistemas criptográficos modernos, como es el sistema RSA. Lo que Gauss conceptualizó como estudio abstracto de congruencias numéricas se ha transformado en el mecanismo esencial que garantiza confidencialidad, integridad y autenticación en el mundo digital.

Lejos de ser una mera curiosidad matemática, el cifrado modular constituye el núcleo operativo de protocolos de seguridad que utilizamos constantemente sin ser consciente de ellos. Como señalan Rivest, Shamir y Adleman -padres fundadores del sistema de cifrado RSA- en su artículo fundacional del RSA, “el método se basa en la exponenciación modular y la dificultad de factorizar números grandes” (Rivest et al., 1978). Desde las transacciones bancarias en línea hasta los mensajes cifrados de aplicaciones de mensajería, pasando por las firmas digitales en documentos oficiales, la criptografía modular trabaja incansablemente en segundo plano.

Sistemas como el RSA -precisamente el que implementamos en esta práctica- aprovechan propiedades contraintuitivas de la exponenciación modular para crear esquemas donde descifrar sin la clave correcta resulta computacionalmente inviable. William Stallings destaca que “la seguridad del RSA descansa en la dificultad computacional de factorizar el módulo n cuando este es producto de dos primos grandes” (Stallings, 2018).

Esta transformación de conceptos matemáticos puros en herramientas de seguridad practica ejemplifica como las abstracciones teóricas pueden convertirse en guardianes esenciales de nuestra privacidad en la era digital, demostrando que la elegancia matemática puede traducirse en protección tangible para la información sensible.

En el contexto de la criptografía aplicada se desarrolla esta práctica, cuyo objetivo principal es implementar sistemas de cifrado seguro mediante la aplicación de conceptos de aritmética modular, demostrando su utilidad práctica en el ámbito de la seguridad informática y afianzando los fundamentos matemáticos que sustentan la criptografía de clave pública.

Como objetivos secundarios de esta práctica encontramos:

- Profundizar en el desarrollo de código en el entorno de programación de Python, optimizando una librería de funciones, creada en la práctica anterior, para el cálculo de conceptos matemáticos avanzados,
- Diseñar e implementar algoritmos eficientes para operaciones criptográficas basadas en aritmética modular, optimizando el rendimiento computacional,
- Construir interfaces de usuario funcionales para sistemas de comunicación cifrada, manejando entradas y salidas por terminal de forma robusta,

- Implementar mecanismos exhaustivos de gestión de errores para operaciones criptográficas y validación de entradas,
- Garantizar la fiabilidad del sistema mediante pruebas automatizadas con la suite pytest y casos de prueba comprehensivos,
- Fomentar el trabajo colaborativo.

El proyecto consiste en la implementación de un sistema criptográfico integral compuesto por tres módulos .py principales:

1. **rsa.py**: librería criptográfica que implementa el algoritmo RSA completo, incluyendo generación de pares de claves, funciones de padding, cifrado y descifrado de mensajes, y mecanismos de ataque criptográfico, utilizando como base matemática las funciones de modular.py.
2. **registrarusuario.py**: módulo de gestión de identidades que permite crear nuevos usuarios en el sistema, generando automáticamente pares de claves RSA y almacenándolos en archivos seguros dentro del directorio "Usuarios".
3. **criptochat.py**: aplicación principal de chat cifrado de extremo a extremo que utiliza las claves generadas para permitir comunicación segura entre usuarios, implementando tanto el cifrado con clave pública del destinatario como el descifrado con clave privada local.

DISEÑO E IMPLEMENTACIÓN

1. Arquitectura del código

La arquitectura del sistema implementa una estructura modular estratificada que establece una clara división de responsabilidades entre componentes especializados. Este enfoque facilita el mantenimiento evolutivo, el aislamiento de fallos y la escalabilidad futura. Cada módulo opera con autonomía funcional, permitiendo optimizaciones específicas sin comprometer la estabilidad global del ecosistema de criptografía.

El sistema se articula alrededor de tres pilares fundamentales que interactúan de forma sinérgica:

a. `rsa.py`: el motor criptográfico central

Este módulo constituye el núcleo algorítmico del sistema, implementando las operaciones fundamentales del cifrado asimétrico RSA mediante cuatro capas funcionales:

- Generación de identidades digitales

Sistema para crear pares de claves criptográficas mediante selección probabilística de parámetros primos y cálculo de relaciones matemáticas entre exponentes. Este proceso combina algoritmos de búsqueda de números primos dentro de rangos especificados con el cálculo de inversos modulares para establecer la relación fundamental entre claves privadas y públicas. La robustez del sistema depende críticamente de la calidad de los primos generados y de la correcta aplicación de las identidades matemáticas que garantizan la reversibilidad de las operaciones criptográficas.

- Mecanismos de ofuscación segura

Protocolos de enmascaramiento que introducen aleatoriedad controlada en los procesos de cifrado, transformando esquemas deterministas en probabilísticos para fortalecer la resistencia criptográfica. Estos mecanismos implementan técnicas de padding que añaden ruido matemático a los mensajes antes del cifrado, eliminando patrones predecibles y previniendo ataques basados en análisis de similitud entre textos cifrados. La aleatoriedad introducida es eliminada durante el descifrado mediante operaciones determinísticas que preservan el contenido original.

- Transformaciones de datos

Conjunto de operaciones matemáticas que permiten la transición bidireccional entre información legible y representaciones cifradas, preservando la integridad semántica mediante transformaciones reversibles. Este proceso incluye la codificación de texto en formatos numéricos adecuados para operaciones criptográficas, la aplicación de funciones matemáticas de cifrado que protegen la confidencialidad, y la posterior reconstrucción del contenido original mediante operaciones inversas -cuidadosamente diseñadas-.

- Análisis de vulnerabilidades

Herramientas de “auditoría” criptográfica que demuestran puntos débiles en implementaciones teóricas, facilitando la comprensión de requisitos de seguridad en sistemas reales. Estas capacidades incluyen métodos para evaluar la fortaleza de las claves generadas, identificar debilidades en esquemas de protección, y demostrar ataques prácticos contra configuraciones inseguras, sirviendo como instrumento educativo para comprender los principios fundamentales de la seguridad criptográfica.

b. registrarusuario.py: la capa de gestión de identidades digitales

El archivo registrarusuario.py funciona como interfaz entre el usuario y el sistema de generación de claves de rsa.py, permitiendo la creación de identidades criptográficas de manera interactiva mediante entrada por terminal. La arquitectura de este código se basa en patrones de configuración guiada, simplificando el proceso de establecimiento de parámetros de seguridad, basándose en las siguientes funcionalidades de manejo:

- Configuración de perfiles criptográficos

Proceso interactivo para establecer parámetros de seguridad personalizados según requisitos específicos de cada contexto de uso. El sistema guía al usuario en la selección de rangos de números primos y configuración de parámetros de padding, adaptando el nivel de seguridad a las necesidades particulares de cada caso.

- Persistencia segura de credenciales

Mecanismos de almacenamiento estructurado que mantiene la separación entre elementos públicos y privados, garantizando la confidencialidad de información sensible. La implementación utiliza un esquema de archivos separados para claves públicas y privadas.

- Validación de parámetros de seguridad

Verificaciones automáticas que aseguran la coherencia entre configuraciones seleccionadas y estándares criptográficos establecidos. El sistema comprueba la validez de los rangos de primos especificados y la adecuación del padding configurado.

El procesamiento del registro sigue una secuencia definida por:

1. Recepción y validación inicial: verifica la estructura básica de los datos ingresados, comprobando que los valores numéricos sean enteros positivos y que el nombre de usuario sea válido y no usado previamente.
2. Análisis y procesamiento de parámetros: separa y procesa los diferentes componentes de la configuración y los prepara para la generación criptográfica.
3. Generación y enrutamiento criptográfico: Determina los algoritmos adecuados para la generación de claves y aplica las transformaciones matemáticas necesarias para crear los pares de claves de RSA.
4. Control de errores y validación: implementa un mecanismo que captura excepciones durante la generación de claves, diferenciando entre errores matemáticos (como la imposibilidad de encontrar primos) y errores del sistema.
5. Formateo de salida: estructura los resultados en formatos estandarizados para su almacenamiento en archivos, adecuándose a las especificaciones de formato requeridas.

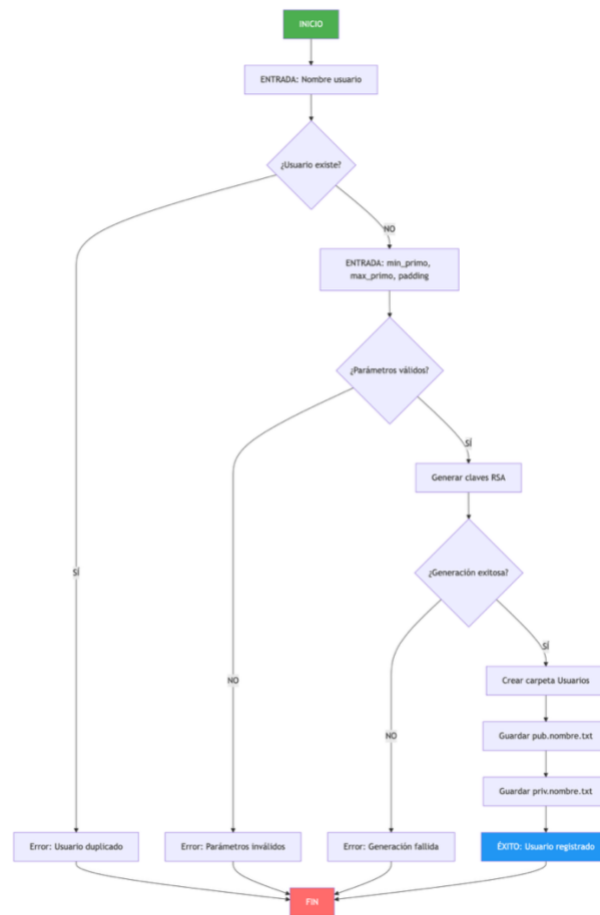


Figura 1. Diagrama de flujo del proceso de registro de usuarios. Elaboración propia.

El diseño de registrarusuario.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice eficiencia, mantenibilidad y robustez en el sistema de gestión de identidades criptográficas:

- Algoritmos optimizados: implementamos generación eficiente de claves RSA mediante selección inteligente de primos y calculo rápido de inversos modulares, aprovechando propiedades matemáticas de la aritmética modular.
- Detección precoz: identificamos tempranamente configuraciones invalidas como rangos de primos incorrectos o parámetros de seguridad inconsistentes, evitando procesos de generación costosos cuando están condenados al fallo.
- Validación en cascada: aplicamos verificaciones progresivas que descartan entradas erróneas en cuanto se detecta el primer problema, optimizando el tiempo de procesamiento y mejorando la experiencia del usuario.
- Manejo eficiente de recursos: utilizamos estructuras de datos ligeras para la gestión de parámetros y empleamos técnicas de escritura secuencial que minimizan la sobrecarga de memoria durante la persistencia de archivos.

c. `criptochat.py`: la plataforma de comunicación cifrada

El archivo `criptochat.py` funcionan como interfaz principal entre los usuarios y el sistema de cifrado RSA, permitiendo la comunicación segura mediante sesiones interactivas de chat por terminal. La arquitectura de este código se basa en patrones de conversación cifrada, simplificando el proceso de intercambio de mensajes protegidos, basándose en las siguientes funcionalidades de manejo:

- Gestión de sesiones criptográficas

Sistema que establece y mantiene contextos seguros de comunicación entre usuarios identificados. La aplicación carga automáticamente las claves públicas y privadas correspondientes, configurando los parámetros de seguridad específicos para cada participante del chat.

- Interfaz de operaciones criptográficas

Entorno interactivo que proporciona modos especializados para cifrado y descifrado, permitiendo al usuario alternar entre la protección de mensajes salientes (usando claves públicas del destinatario) y la interpretación de mensajes entrantes (usando claves privadas locales).

- Procesamiento adaptativo de mensajes

Mecanismos inteligentes que detectan y procesan automáticamente los diferentes formatos de comunicación, tanto texto plano legible como listas de enteros cifrados, aplicando las transformaciones criptográficas apropiadas en cada caso.

El procesamiento del registro sigue una secuencia definida por:

1. Inicialización y Carga de Claves: Verifica la disponibilidad de los archivos de claves y carga los parámetros criptográficos necesarios para ambos usuarios participantes.
2. Presentación de Menú Interactivo: Ofrece al usuario las opciones disponibles (cifrar, descifrar, salir) y captura su selección de manera robusta.
3. Procesamiento de Operaciones: Dirige el flujo hacia las funciones específicas de cifrado o descifrado según la elección del usuario, aplicando las transformaciones matemáticas correspondientes.
4. Manejo de Errores y Validación: Implementa un sistema que captura excepciones durante las operaciones criptográficas, diferenciando entre errores matemáticos y problemas del sistema.
5. Formateo y Presentación de Resultados: Adapta los outputs al formato requerido, mostrando texto plano legible o listas de enteros cifrados según corresponda.

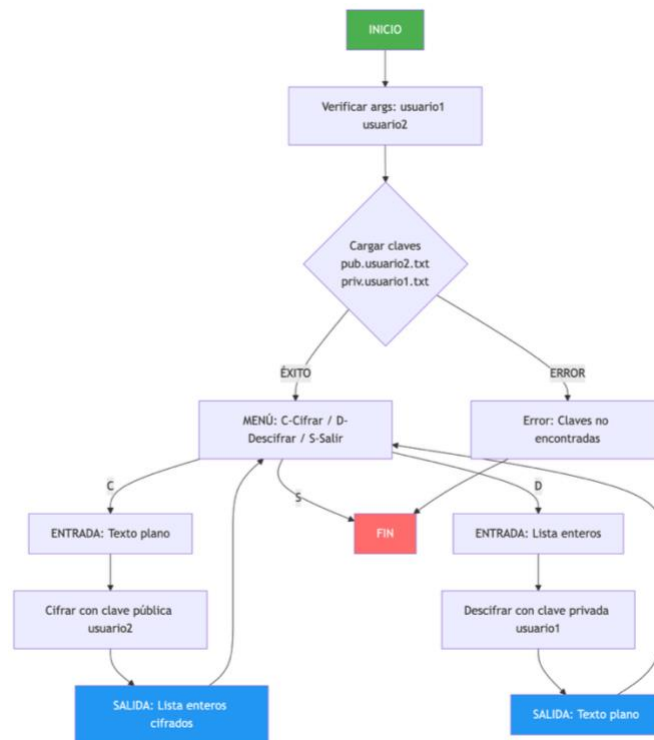


Figura 2. Diagrama de flujo del proceso de comunicación cifrada. Elaboración propia.

El diseño de `criptochat.py` incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice una rápida comunicación en tiempo real:

1. Gestión Eficiente de Recursos: Utiliza estructuras de datos optimizadas para el manejo de mensajes y claves, minimizando la sobrecarga de memoria durante sesiones prolongadas.
2. Validación en Tiempo Real: Verifica instantáneamente la validez de las entradas del usuario, proporcionando retroalimentación inmediata sobre errores de formato o parámetros inválidos.
3. Procesamiento por Lotes: Optimiza el manejo de mensajes largos mediante técnicas de segmentación y procesamiento por bloques, manteniendo la tasa de respuesta del sistema.
4. Manejo Robusto de Archivos: Implementa mecanismos de lectura/escritura eficientes para la gestión de claves, asegurando la disponibilidad constante de los recursos criptográficos.

2. Algoritmos principales

El núcleo del sistema RSA implementado reside en la aplicación eficiente de teoremas fundamentales de la teoría de números y la aritmética modular. Cada funcionalidad ha sido programada con un enfoque dual: profundizar en sus bases matemáticas para garantizar corrección absoluta, mientras se exploran optimizaciones algorítmicas que reducen sustancialmente la complejidad computacional y los tiempos de ejecución. El diseño prioriza la aplicabilidad práctica en escenarios reales, donde el balance entre seguridad y rendimiento resulta crítico.

En este apartado analizaremos exhaustivamente cada algoritmo implementado, justificando las decisiones de diseño tomadas en función de propiedades matemáticas específicas, así como el flujo de ejecución que asegura tanto la robustez criptográfica como la eficiencia operativa del sistema completo:

- `generar_claves(min_primo :int, max_primo :int) -> Tuple[int, int, int]:`

El algoritmo implementado para la generación de claves RSA se basa en principios criptográficos establecidos que garantizan la seguridad mediante la dificultad computacional de factorizar números grandes (Rivest et al., 1978). La función opera mediante un proceso estructurado en tres fases fundamentales que aseguran la robustez del par de claves generado.

Iniciamos con la selección probabilística de primos, donde obtenemos dos números primos distintos dentro del rango especificado mediante muestreo aleatorio de una lista precomputada. Esta aproximación asegura la distribución uniforme de los primos seleccionados y evita sesgos en la generación. La independencia entre p y q es crucial para garantizar la fortaleza criptográfica del módulo n resultante.

Procedemos con el cálculo de parámetros derivados, donde computamos $\phi(n) = (p-1)(q-1)$ utilizando la propiedad multiplicativa de la función ϕ de Euler para números primos. Esta optimización evita el cálculo directo de $\phi(n)$ para n grande, reduciendo significativamente la complejidad computacional. Posteriormente, seleccionamos el exponente público e mediante una estrategia escalonada que prioriza valores criptográficamente eficientes como 65537, 257 o 17, verificando en cada paso la coprimalidad con $\phi(n)$.

Finalizamos con la derivación del exponente privado mediante la identidad de Bézout extendida, que nos permite calcular d como el inverso modular de e módulo $\phi(n)$. Implementamos este cálculo de forma iterativa para garantizar estabilidad numérica y aplicamos ajustes modulares para asegurar que d sea positivo y se encuentre en el rango adecuado.

Hemos optado por una implementación iterativa en todas las fases del proceso, priorizando la eficiencia de memoria y el rendimiento en la ejecución. Esta decisión es particularmente relevante cuando se trabajan con rangos amplios de números primos, donde el overhead de llamadas recursivas podría impactar significativamente el tiempo de generación.

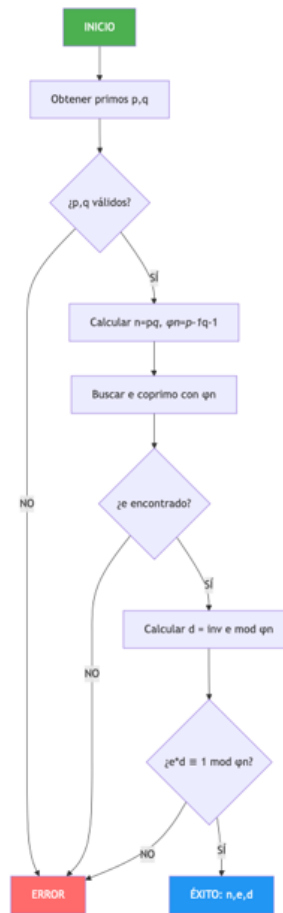


Figura 3. Diagrama de flujo de la función `generar_claves(min_primo :int, max_primo :int) -> Tuple[int, int, int]`. Elaboración propia.

El algoritmo destaca por su eficiencia en la generación de claves de tamaño pequeño a moderado, siendo capaz de producir pares seguros en rangos típicos de 10^3 a 10^6 . Para números extremadamente grandes, el rendimiento está dominado por la eficiencia de las funciones subyacentes de verificación de primalidad y cálculo de inversos modulares.

- `aplicar_padding(m: int, digitos_padding: int)->int:`

El objetivo de esta función es añadir, dado un número dígitos de padding, tantos números aleatorios como dígitos de padding al mensaje ya cifrado. El padding es una técnica empleada en numerosos ámbitos, entre los que se encuentra la criptografía. El objetivo principal de esta técnica es conseguir que el último bloque del mensaje cifrado, que en ocasiones puede ser más corto o largo que el tamaño predeterminado, contenga el mismo número de bits que los fijados por los algoritmos de cifrado.

De esta manera, al añadir dígitos mediante padding, logramos que el sistema sea menos vulnerable a ataques de diccionario.

El algoritmo se implementa mediante un bucle principal `for` que será el encargado de añadir los números aleatorios en función de los dígitos de padding. Distinguimos el caso en el que el número de dígitos de padding es 0, devolviendo el número tal cual nos lo dieron en el argumento de la función.

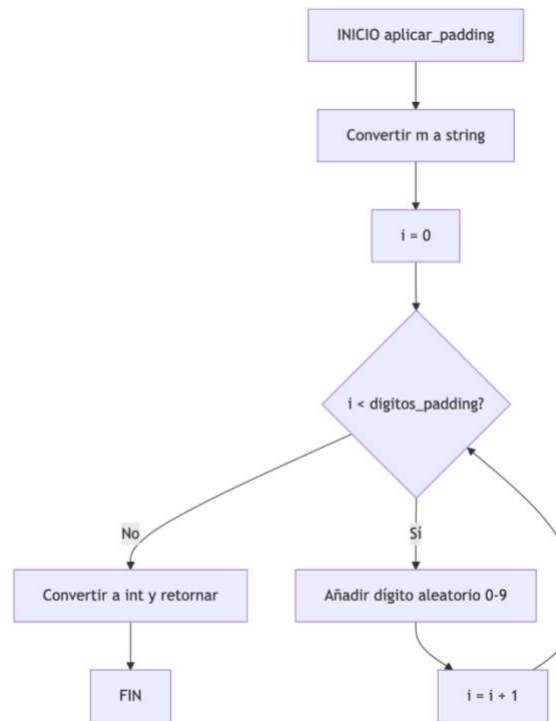


Figura 4. Diagrama de flujo de la función aplicar_padding(m: int, digitos_padding: int)->int Elaboración propia.

El diagrama de flujo deja constancia del bucle principal, donde vemos cómo se van añadiendo, si los `digitos_padding` son mayores que 0, números aleatorios al número inicial hasta que finalmente devolvemos el número con el padding añadido. En caso de que `digitos_padding` sea 0, devolvemos `m`.

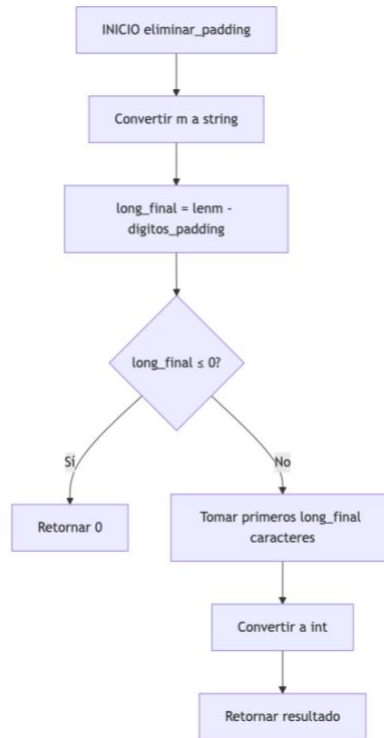
- `eliminar_padding(m: int, digitos_padding: int)->int`:

Esta función sigue la misma lógica que la anterior, pero si lo que queríamos antes era añadir un padding para ajustarlo al número de bits del algoritmo de cifrado, esta vez queremos eliminarlo para descifrar el mensaje recibido.

Por tanto, recibimos el número de dígitos de padding (que es una información pública) y el mensaje al que queremos eliminarle los números añadidos por padding.

La implementación de la función en este caso tiene en cuenta más casuísticas que se pueden dar al eliminar el padding, teniendo en cuenta los posibles errores que pueda recibir la función, como un número de dígitos de padding mayor a la longitud del mensaje. Por ello, la función `eliminar_padding()` incluye los siguientes casos; el mencionado anteriormente, donde devolvemos un mensaje afirmando que el dato recibido de padding o de mensaje cifrado es erróneo. Cuando el número de dígitos de padding es igual a la longitud del mensaje, en cuyo caso devolvemos 0, ya que no hay mensaje por descifrar, o finalmente, si el número de dígitos de padding es 0, devolvemos el mensaje dado, ya que no hay cifras a eliminar.

En el caso de que no se dé ninguna de estas casuísticas el programa principal está formado por un bucle for el cual añadirá a una variable tipo string (luego se convertirá en numérica) los valores reales del mensaje sin padding. Esta ejecución la llevamos a cabo identificando primero cuál es la longitud real del mensaje restandole a la longitud del mensaje los dígitos de padding, y, una vez hallada, ejecutaremos el for.



- Figura 5. Diagrama de flujo de la función eliminar_padding(m: int, digitos_padding: int)->int. Elaboración propia.

El diagrama de flujo muestra de forma clara la primera evaluación de los dígitos de padding respecto a la longitud del mensaje. En caso de que los dígitos_padding sea iguales o mayores a la longitud del mensaje, devolvemos 0, ya que no quedaría nada del mensaje. En caso contrario, calculamos m sin padding realizando un bucle for que irá desde 0 hasta len(m)-digitos_padding añadiendo a num_final los números correctos del mensaje sin padding, convertimos el número a tipo entero y lo devolvemos.

- cifrar_rsa(m :int, n : int, e :int, digitos_padding :int) -> int:

El algoritmo implementado para el cifrado RSA se basa en el principio criptográfico establecido de seguridad mediante exponenciación modular (Rivest et al., 1978). La función opera mediante un proceso estructurado que garantiza la confidencialidad del mensaje mediante transformaciones matemáticas irreversibles sin la clave privada.

Iniciamos con la aplicación de padding criptográfico, donde añadimos dígitos aleatorios al mensaje original según el parámetro digitos_padding especificado. Esta técnica transforma el esquema determinista en probabilístico, asegurando que mensajes idénticos produzcan cifrados diferentes y previniendo ataques basados en análisis de patrones.

Procedemos con la validación del tamaño del mensaje, verificando que el mensaje con padding no exceda el módulo n. Implementamos un mecanismo de ajuste automático que garantiza la compatibilidad incluso en casos extremos, manteniendo la integridad semántica del proceso de cifrado.

Finalizamos con la exponenciación modular segura, aplicando la transformación criptográfica $c = m^e \bmod n$ mediante algoritmos de cuadrados repetidos. Esta optimización reduce la complejidad computacional de $O(n)$ a $O(\log n)$, permitiendo el manejo eficiente de exponentes grandes típicos en configuraciones RSA seguras.

Hemos optado por una implementación que prioriza la robustez frente a condiciones edge cases, incluyendo manejo específico para padding cero y verificación de límites modulares. La función garantiza que cualquier mensaje válido dentro de los parámetros especificados pueda ser cifrado correctamente.

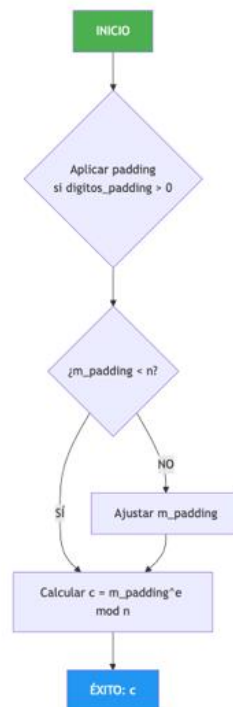


Figura 6. Diagrama de flujo de la función `cifrar_rsa(m :int, n :int, e :int, digitos_padding :int) -> int`. Elaboración propia.

El algoritmo demuestra eficiencia particular en el manejo de mensajes de tamaño moderado con configuraciones de padding típicas (1-5 dígitos). El rendimiento está dominado por la operación de exponenciación modular, optimizada mediante el algoritmo de cuadrados repetidos que asegura tiempos de respuesta adecuados incluso para módulos grandes.

- `descifrar_rsa(c :int, n :int, d :int, digitos_padding :int) -> int`:

El algoritmo implementado para el descifrado RSA se fundamenta en la propiedad matemática de reversibilidad de la exponenciación modular cuando se utiliza el exponente privado correcto (Katz & Lindell, 2015). La función ejecuta una secuencia precisa de operaciones que recuperan el mensaje original a partir del texto cifrado.

Iniciamos con la exponenciación modular inversa, aplicando la transformación $m_{\text{padded}} = c^d \bmod n$ mediante el algoritmo de cuadrados repetidos. Esta operación revierte matemáticamente el proceso de cifrado, aprovechando la relación fundamental $e \cdot d \equiv 1 \bmod \phi(n)$ que garantiza la recuperación exacta del mensaje con padding.

Procedemos con la eliminación del padding criptográfico, removiendo los dígitos aleatorios añadidos durante el cifrado según el parámetro `digitos_padding` especificado. Esta etapa restaura el mensaje numérico original mediante un proceso determinístico que preserva completamente la información semántica inicial.

Finalizamos con la validación de la integridad del resultado, asegurando que el mensaje descifrado se encuentra dentro de los rangos esperados y mantiene coherencia con los parámetros criptográficos establecidos. Esta verificación proporciona garantías adicionales sobre la corrección del proceso.

Hemos implementado un enfoque que enfatiza la robustez frente a entradas potencialmente corruptas o manipuladas, incluyendo mecanismos de verificación que detectan inconsistencias antes de proceder con operaciones costosas computacionalmente.

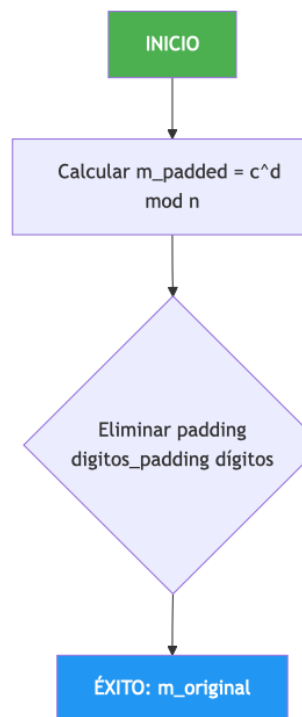


Figura 7. Diagrama de flujo de la función `descifrar_rsa(c:int, n:int, d:int, digitos_padding:int) -> int`. Elaboración propia.

El algoritmo demuestra eficiencia óptima en el descifrado de mensajes, con un rendimiento dominado por la operación de exponenciación modular. La implementación aprovecha las mismas optimizaciones de cuadrados repetidos utilizadas en el cifrado, asegurando tiempos de respuesta consistentes para diversos tamaños de módulo.

- `codificar_cadena(s:str)->List[int]`:

El objetivo de este algoritmo es convertir una cadena de caracteres en una lista de enteros cuyos valores son los valores ASCII de las letras y caracteres asociados al mensaje. Este es el paso inicial para cifrar la cadena, por ello iremos recorriendo la cadena y añadiendo a una lista los valores ASCII de las letras contenidas en el mensaje pasado como argumento. Finalmente, devolvemos la lista.

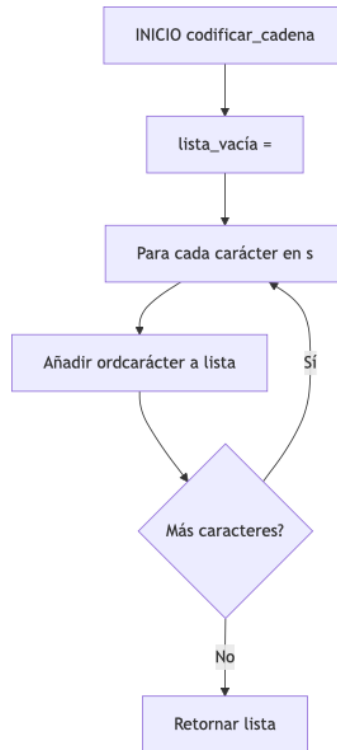


Figura 8. Diagrama de flujo de la función `codificar_cadena:(s: str)->List[int]`. Elaboración propia.

Como podemos ver en el diagrama de flujo, primero inicializamos la lista vacía. El programa se compone de un bucle principal `for` que irá evaluando cada carácter del mensaje y añadiendo a una lista el valor ASCII de dicho carácter. Finalmente, devuelve la lista.

- `decodificar_cadena(m: List[int]) -> str:`

Esta función recibe una lista de enteros y decodifica los valores de la lista convirtiéndolos en caracteres para poder devolver el mensaje ya descifrado. Este es el último paso cuando desciframos un mensaje, ya que es el que nos permite leer el mensaje escrito inicialmente antes de ser cifrado. Sabiendo que para codificar la cadena habíamos usado los valores ASCII de las letras, esta vez haremos lo mismo solo que con los valores ASCII de los números, devolviendo así en la variable `mensaje`, el contenido descifrado por esta función.

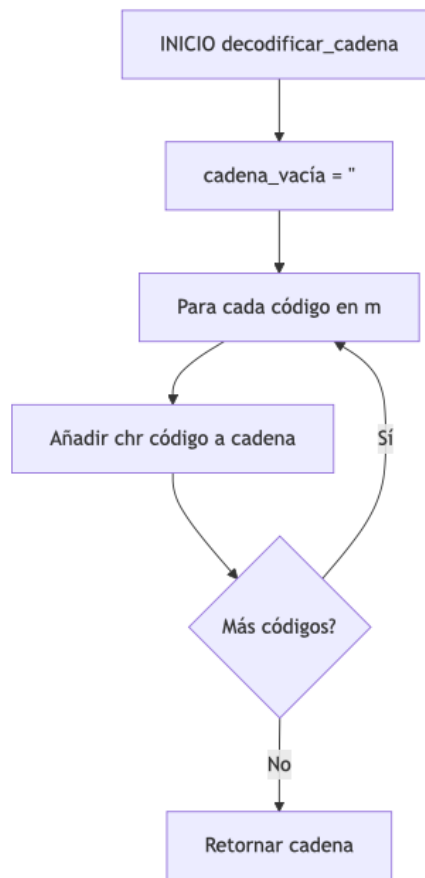


Figura 9. Diagrama de flujo de la función `decodificar_cadena(m: List[int]) -> str`. Elaboración propia.

Como podemos ver en el diagrama de flujo, inicializamos la variable vacía. El programa se compone de un bucle principal `for` que irá evaluando cada valor de la lista y añadiendo a una variable el valor carácter (`chr`) de ese carácter. Finalmente, devuelve la variable.

- `cifrar_cadena_rsa(s: str, n: int, e: int, digitos_padding: int) -> List[int]`:

El algoritmo programado en esta función tiene como meta cifrar un mensaje dada una cadena de caracteres. Recibe como argumentos una cadena, la clave pública `n`, `e` y dígitos de padding. Esto es porque el proceso de cifrado sigue la siguiente lógica; primeramente, codificaremos la cadena, para convertir los caracteres del mensaje en valores numéricos, una vez codificada la cadena quedará cifrarla. Iremos cifrando cada valor de la lista mediante la función `cifrar_rsa()` que, como se ha explicado anteriormente, se encargará primero de añadirle el número de dígitos de padding (utilizando la función `aplicar_padding()`) establecidos por el usuario. Una vez añadidos estos números aleatorios solo quedará cifrar la cadena. Este proceso se llevará a cabo únicamente utilizando la clave pública y no la privada, porque el cifrado de mensajes es un proceso que puede hacer todo el mundo, porque lo que nos interesa no es que se cifren mensajes si no que aquellas personas que no conozcan la clave privada sean capaces de descifrar el mensaje enviado. Cifrado el carácter se añadirá a la lista que contiene el mensaje cifrado y se devolverá.

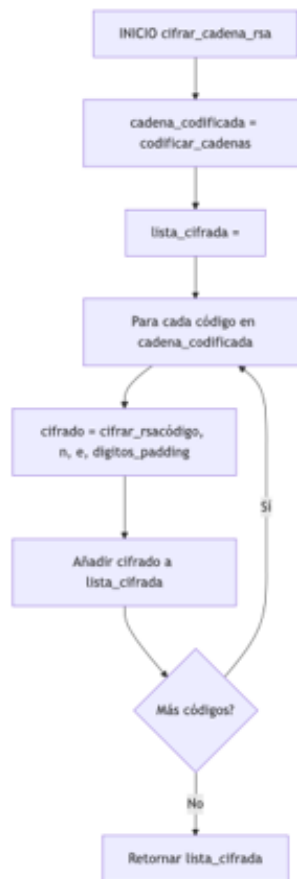


Figura 10. Diagrama de flujo para la función `coprimos(n: int, m: int) -> bool`. Elaboración propia

Como podemos observar, el siguiente diagrama de flujo describe como primero se inicializa la lista y se codifica la cadena pasada como argumento a la función y como luego, mediante un for se irán cifrando los caracteres y se añadirán a la lista que contendrá el mensaje cifrado.

- `descifrar_cadena_rsa(cList: List[int], n: int, d: int, digitos_padding: int) -> str`:

Este algoritmo se encarga de, dado un mensaje cifrado con RSA descifrar su contenido. La función recibe como argumentos la lista con los caracteres del mensaje, el módulo de RSA, la clave privada y el número de dígitos de padding.

En este caso, el proceso que se llevará a cabo será el contrario al ejecutado en la función de cifrado. Primero llamaremos a la función `descifrar_rsa` que se encargará primero de eliminar los dígitos de padding del mensaje y luego usando la clave privada y el módulo devolver los valores numéricos asociados al mensaje. Como en la función de cifrar hemos codificado primero con los valores ASCII de las letras, lo que nos debería devolver la función es un número de dos o tres cifras, es decir, un valor que se encuentre en las tablas ASCII. Como el programa está ejecutado con un bucle for, iremos descifrando el mensaje carácter a carácter que se irán almacenando primero en la lista `mensaje_limpio` y el mensaje descifrado se irá almacenando en la variable `mensaje_final`. Una vez que tenemos el carácter ASCII de lo que será nuestro carácter del mensaje descifrado, decodificamos el valor llamando a la función `decodificar_cadena`

que nos devolverá el carácter asociado a ese valor numérico. Finalmente, devolveremos el mensaje descifrado.

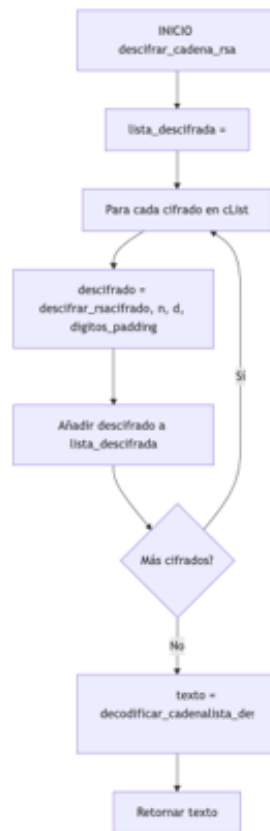


Figura 11. Diagrama de flujo para la función descifrar_cadena_rsa(cList: List[int], n: int, d: int, digitos_padding: int) -> str. Elaboración propia.

El diagrama de flujo deja constancia de la inicialización de la lista y del bucle principal, donde se ve cómo iremos descifrando los valores dados por la lista cList, y añadiéndolos a una lista que luego decodificaremos, devolviendo el mensaje descifrado.

- romper_clave(n: int, e: int) -> int:

La función romper_clave() tiene como objetivo principal tratar de “hackear” un mensaje descifrando su clave privada a través de una clave pública. La función recibe como argumentos la clave pública, siendo n el módulo para RSA y e la clave pública para RSA.

Primero factorizaremos n para encontrar los números primos que la componen haciendo uso de la función factorizar creada en la primera práctica. Llevaremos a cabo una comprobación acerca de los factores que componen n distinguiendo varios casos; el primero, donde n no es producto de exactamente dos primos, en cuyo caso lanzamos el mensaje de que el módulo no es válido. En caso de que no se de esta situación, obtendremos los factores primos, revisando siempre que n esté compuesto únicamente por dos primos distintos, si no fueran distintos, lanzamos un mensaje. Verificaremos que los primos tengan exponente 1, ya que, si no, el cálculo no es válido. Finalmente, si no es necesario lanzar mensajes y n es un módulo válido, realizaremos la operación que nos permitirá obtener la clave privada. Empezaremos calculando phi

de n como producto de $(p-1) * (q-1)$ siendo p y q los factores primos que componen n . Ahora seguiremos la fórmula $d * e \equiv 1 \pmod{\Phi(n)}$, donde d es la clave privada, e es la clave pública y el módulo es ϕ de n . Calculada la ϕ , verificaremos que e y ϕ de n sean coprimos, lanzando un mensaje en caso de que no lo sean. Realizada esta comprobación, resolveremos la ecuación congruente calculando la inversa de e módulo ϕ de n , lanzando un mensaje en caso de que no exista la inversa de e . Finalmente, llevamos a cabo una comprobación final comprobando que $e * d$ da resto 1 al hacerlo módulo ϕ de n , devolviendo d si el cálculo es correcto y devolviendo un mensaje de error en caso contrario.

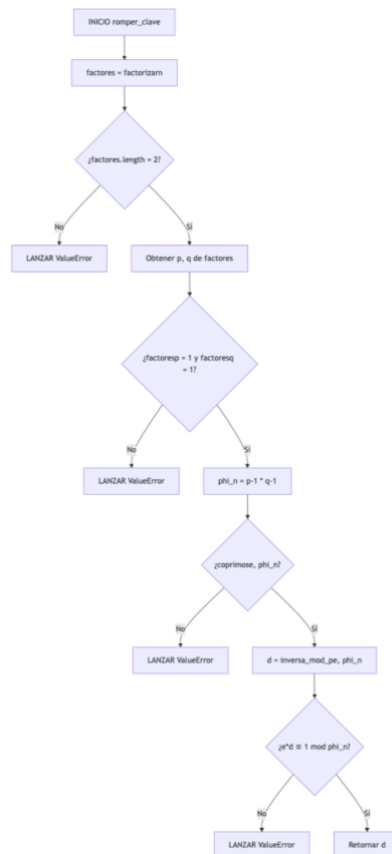


Figura 12. Diagrama de flujo de la función `romper_clave(n: int, e: int) -> int`. Elaboración propia.

Observamos aquí la estructura del algoritmo: realizamos las primeras comprobaciones; si n tiene exactamente dos factores primos, devolviendo Error si no es así y pasando a la siguiente comprobación si no; obtener los dos factores primos distintos, devolviendo Error si no son exactamente dos factores primos distintos y pasando a la siguiente comprobación en caso contrario; ver si cada factor tiene exponente 1, devolviendo Error en caso de que no sea así y pasando al cálculo de d si tienen exponente 1. Calculamos ϕ de n y verificamos que e y ϕ_n sean coprimos, devolviendo Error si no lo son y calculando la inversa de e en caso de que sí lo sean. Devolveremos Error si no existe inversa de e y calcularemos d en caso contrario. Realizamos la verificación final donde comprobamos si al ejecutar $(e * d) \% \phi_n = 1$, si lo es, devolvemos d , si no, Error.

- `ataque_texto_elegido(cList: List[int], n: int, e: int) -> str:`

El objetivo de esta función es dado un mensaje y una clave pública, ‘hackear’ un mensaje dado y descubrir así su contenido.

Para ello empleamos el ataque de diccionario. Ataque que solo podemos emplear cuando el padding es 0 y que hará que de forma mucho más eficiente seamos capaces de descifrar mensajes largos. En caso de que el padding no sea 0 deberemos configurar la función obteniendo la clave privada y con ese dato ya descifrar el mensaje. Sin embargo, nuestro caso se da con padding 0. El hecho de que sea más eficiente se da porque, en vez de ir descifrando carácter a carácter, tenemos un diccionario cuyas claves son caracteres cifrados de tal manera que solo tendríamos que identificar el carácter que vamos a descifrar en ese momento y asignarle el valor asociado a la clave.

Para obtener el mensaje, creamos un diccionario que contenga todos los caracteres existentes, tanto los incluidos en UTF-8 como los incluidos en las tablas ASCII, es decir, números, espacios en blanco, mayúsculas y minúsculas, signos de puntuación, tildes...

Al crear este diccionario se simplifica de manera significativa el tiempo de ejecución de esta función, ya que, entonces la función solo tiene que ir comprobando si los caracteres del mensaje dado se encuentran o no en el diccionario (siendo estos, claves). En caso de ser así, como el diccionario ha almacenado como valor de las claves los números de 1 a 255 incluidos, solo nos quedará hacer chr del valor asociado a la clave y eso será añadido a una variable que será la que devolveremos al final de la función. En caso de no encontrarse, lanzaremos un mensaje de error.



Figura 13. Diagrama de flujo para la función ataque_texto_elegido(cList: List[int], n: int, e: int) -> str. Elaboración propia.

El diagrama de flujo deja constancia de la inicialización de la variable que contendrá el mensaje y el diccionario y del primer bucle for, donde se ve cómo iremos cifrando los valores de 0 a 256 y estableciéndolos como clave del diccionario asociándoles el valor i. El siguiente for recorre los valores de la lista dada y comprueba si son claves del diccionario. Si es así añade a la variable chr del valor asociado a la clave caracter y si no devuelve un error. Finalmente, y si no hay errores, devolvemos el mensaje descifrado.

- Ataque avanzado RSA con padding

La implementación de la función `ataque_rsa_avanzado` constituye el desafío opcional de la práctica, representando un esfuerzo por desarrollar una estrategia ofensiva contra sistemas RSA que emplean esquemas de padding básicos. Es crucial destacar que esta funcionalidad se encuentra actualmente en fase avanzada de desarrollo y debugging, habiendo demostrado un funcionamiento parcial, pero requiriendo optimizaciones adicionales para alcanzar una efectividad completa en todos los escenarios.

El sustento teórico del ataque se arraiga en trabajos fundamentales sobre las debilidades de implementaciones RSA, particularmente en la explotación de esquemas de padding determinísticos o predecibles. Como establecieron Bellare y Rogaway en su artículo seminal "Optimal Asymmetric Encryption" (Bellare & Rogaway, 1995) sobre padding óptimo, la seguridad de RSA depende críticamente de la aleatorización efectiva del mensaje antes del cifrado. Nuestro ataque explota precisamente la ausencia de esta aleatorización robusta en el esquema de padding implementado. La relación matemática fundamental que permite el ataque deriva de la estructura del padding: para un mensaje original m y p dígitos de padding, tenemos que $m_con_padding = m \times 10^p + r$, donde $r \in [0, 10^p - 1]$ representa los dígitos aleatorios. Esta formulación, aunque aparentemente simple, establece una restricción crucial: el mensaje original debe satisfacer $m < n / 10^p$. Esta desigualdad, que podría parecer trivial, reduce el espacio de búsqueda en un factor de 10^p , transformando un problema de fuerza bruta sobre $\phi(n)$ en uno sobre $n/10^p$, lo que para valores pequeños de p representa una reducción de órdenes de magnitud en la complejidad computacional.

La estrategia implementada opera mediante un proceso de búsqueda sistemática en múltiples capas, reflejando principios de optimización heurística. Inicialmente, se realiza una búsqueda optimizada en el rango ASCII estándar (0-255), aprovechando que la mayoría de los mensajes de texto emplean caracteres dentro de este conjunto. Para candidato en este rango, se replica exactamente el proceso de cifrado RSA con padding, incluyendo las mismas comprobaciones de modularidad y ajustes implementados en la función `cifrar_rsa`. Esta replicación exacta es crucial, ya que cualquier discrepancia entre el proceso de cifrado original y el recreado durante el ataque invalidaría las comparaciones. Cuando esta búsqueda inicial no produce resultados, el algoritmo activa una segunda fase de búsqueda extendida que explora progresivamente rangos más amplios, manteniendo siempre el límite superior teórico $m < n/10^p$ y empleando el parámetro `max_iteraciones` como mecanismo de contención computacional.

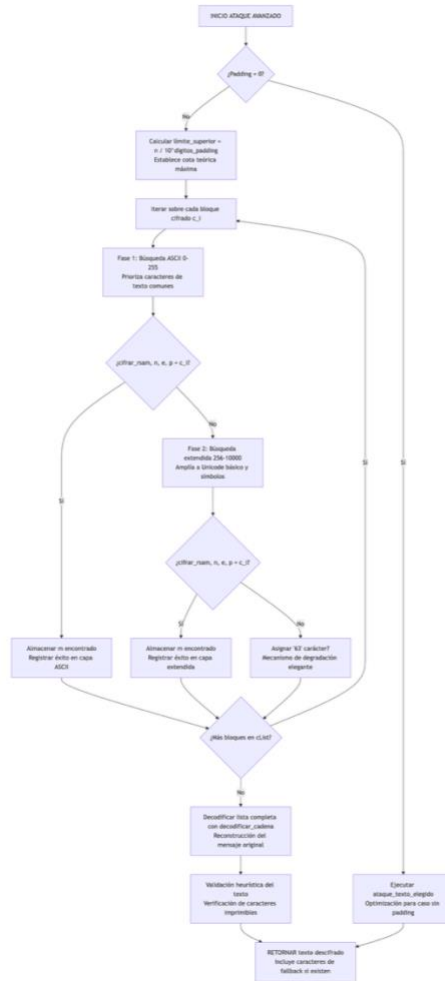


Figura 14. Diagrama de flujo de la función de ataque RSA avanzado con padding. Elaboración propia.

El estado actual de desarrollo presenta un panorama de funcionalidad parcial pero prometedora. El ataque ha demostrado capacidad para descifrar exitosamente mensajes en configuraciones con padding de 1-2 dígitos y módulos de tamaño moderado (hasta 10^5), validando así el enfoque teórico. Sin embargo, enfrenta limitaciones significativas en escenarios más demandantes, donde el crecimiento exponencial del espacio de búsqueda o la presencia de colisiones de padding (diferentes mensajes que producen el mismo texto cifrado) comprometen la efectividad. Estas limitaciones no representan fallos en el concepto sino más bien reflejan las realidades computacionales del criptoanálisis práctico.

Las direcciones de desarrollo futuro se articulan en tres ejes principales de optimización. La paralización masiva del proceso de búsqueda permitiría distribuir la carga computacional mediante técnicas de programación concurrente, particionando el espacio de búsqueda en segmentos manejables procesados independientemente. La implementación de heurísticas sofisticadas, inspiradas en el trabajo “A Mathematical Theory of Communication” (Shannon, 1948) sobre entropía del lenguaje natural, podría priorizar candidatos basándose en frecuencia de caracteres, patrones de bigramas/trigramas, y estructuras sintácticas probables, reduciendo drásticamente el número de intentos necesarios. Finalmente, la integración de técnicas matemáticas avanzadas como variantes del algoritmo baby-step giant-step (Shanks, 1971) adaptadas al contexto específico de RSA con padding, o el uso de optimizaciones basadas en las propiedades multiplicativas de RSA, podría proporcionar mejoras asintóticas significativas.

Este desarrollo, aunque incompleto, encarna perfectamente el espíritu académico de la práctica: no se trata meramente de implementar un ataque operativo, sino de comprender profundamente los mecanismos a través de los cuales elecciones aparentemente menores en la implementación (como el esquema padding) pueden comprometer la seguridad de sistemas criptográficos teóricamente robustos. Como demostró Bleichenbacher en su ataque histórico contra PKCS#1 v1.5 (Bleichenbacher, 2006) y luego Joux en su análisis de esquemas de padding algebraicos (Joux, 2000), la seguridad práctica de RSA reside tanto en la fortaleza de la matemática subyacente como en la elección e implementación de los mecanismos de aleatorización (dentro de que no es del todo aleatorio) y padding. Nuestro trabajo en `ataque_rsa_avanzado` contribuye precisamente a esta comprensión, proporcionando un laboratorio experimental donde teoría y práctica se entrelazan para revelar las sutiles, pero críticas interdependencias entre diseño criptográfico y seguridad implementada.

3. Modificación de algoritmos de modular.py

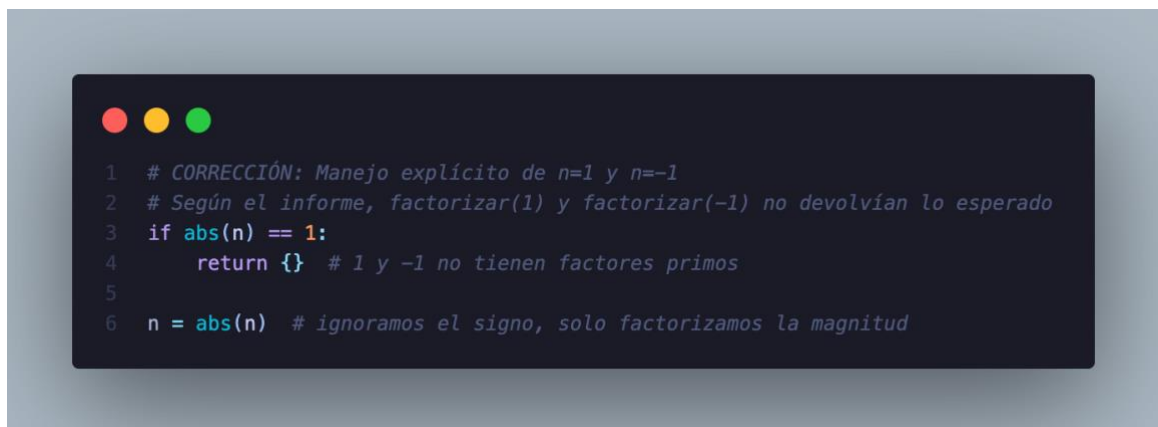
El proceso de refinamiento de la librería modular.py ha involucrado una revisión exhaustiva de cinco funciones críticas que presentaban comportamientos inconsistentes o limitaciones operativas. Estas correcciones responden a la necesidad de garantizar robustez matemática, especialmente en contextos criptográficos donde la precisión en cálculos modulares es fundamental para la seguridad de los sistemas. Las modificaciones implementadas abordan desde el manejo elegante de casos límite hasta la optimización de algoritmos fundamentales, estableciendo un equilibrio entre corrección matemática y eficiencia computacional.

Cada función corregida ha sido analizada bajo tres dimensiones críticas: completitud en el manejo de casos especiales, consistencia en los valores de retorno según especificaciones matemáticas, y eficiencia en la ejecución de algoritmos subyacentes.

- `factorizar(n: int) -> Dict[int, int]`:

El problema inicial residía en que la implementación inicial no manejaba correctamente los casos límite de números unitarios, devolviendo resultados inconsistentes para $n = 1$ y $n = -1$ según las especificaciones documentadas.

Se implementó un manejo explícito de los casos unitarios mediante una verificación temprana que intercepta los valores $|n| = 1$, retornando un diccionario vacío en concordancia con la definición matemática establecida. Esta optimización evita procesamiento innecesario para casos triviales y garantiza coherencia en los valores de retorno.



```
1  # CORRECCIÓN: Manejo explícito de n=1 y n=-1
2  # Según el informe, factorizar(1) y factorizar(-1) no devolvían lo esperado
3  if abs(n) == 1:
4      return {} # 1 y -1 no tienen factores primos
5
6  n = abs(n) # ignoramos el signo, solo factorizamos la magnitud
```

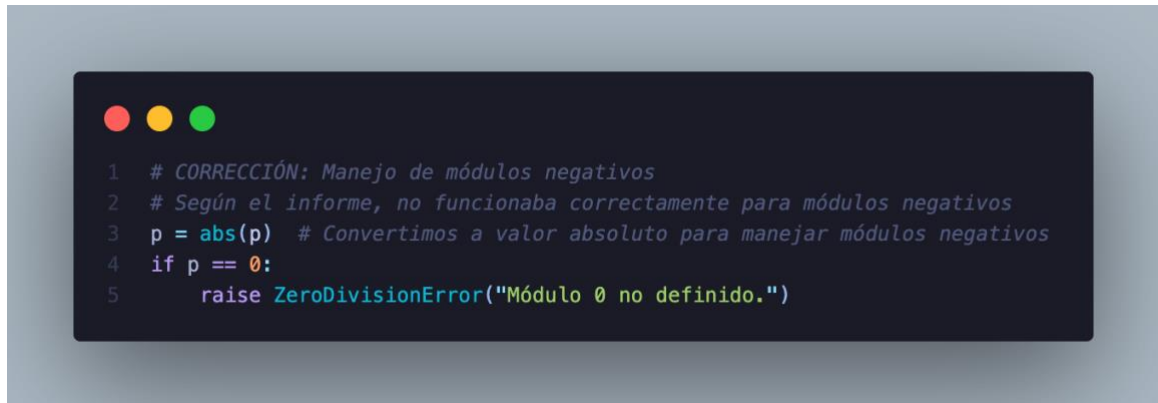
Figura 15. Modificación del código de `factorizar (n: int) -> Dict[int, int]`. Elaboración propia

- `potencia_mod_p(base: int, exp: int, p: int) -> int`:

El problema inicial residía en que la función presentaba comportamiento indefinido para módulos negativos y no manejaba adecuadamente el caso de exponentes negativos en contextos modulares.

Se incorporó normalización de módulos mediante valor absoluto para garantizar consistencia en el cálculo modular, junto con una extensión para manejar exponentes negativos mediante el cálculo de inversos

modulares. Esta mejora amplía la aplicabilidad de la función a escenarios criptográficos donde los exponentes negativos son relevantes.



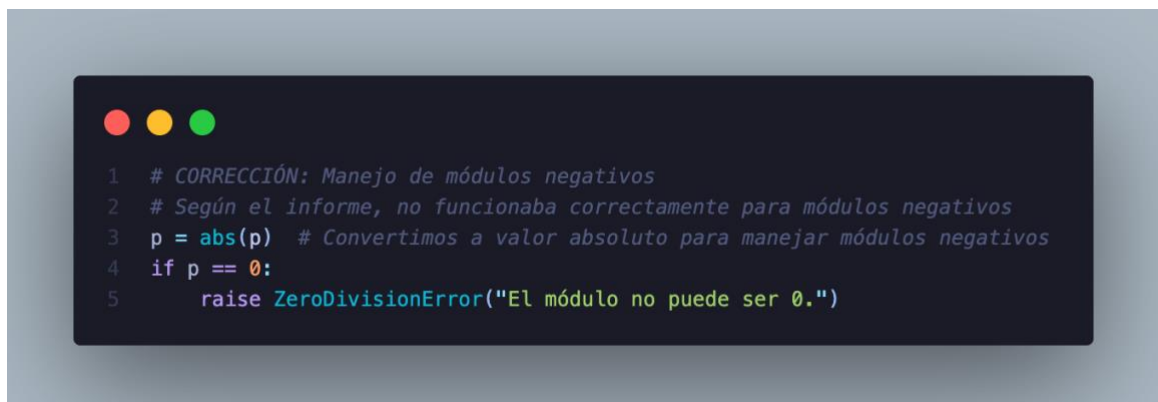
```
1 # CORRECCIÓN: Manejo de módulos negativos
2 # Según el informe, no funcionaba correctamente para módulos negativos
3 p = abs(p) # Convertimos a valor absoluto para manejar módulos negativos
4 if p == 0:
5     raise ZeroDivisionError("Módulo 0 no definido.")
```

Figura 16. Modificación del código de potencia_mod_p (base: int, exp: int, p: int) -> int. Elaboración propia.

- inversa_mod_p (n: int, p: int) -> int:

El problema residía en la incapacidad para procesar módulos negativos y falta de robustez en el algoritmo extendido de Euclides para casos límite.

Hemos implementado un enfoque iterativo del algoritmo extendido de Euclides que garantiza estabilidad numérica y correcto manejo de módulos negativos mediante normalización. La implementación incluye verificación explícita de invertibilidad y asegura que el resultado permanezca en el rango canónico $[0, p-1]$.



```
1 # CORRECCIÓN: Manejo de módulos negativos
2 # Según el informe, no funcionaba correctamente para módulos negativos
3 p = abs(p) # Convertimos a valor absoluto para manejar módulos negativos
4 if p == 0:
5     raise ZeroDivisionError("El módulo no puede ser 0.")
```

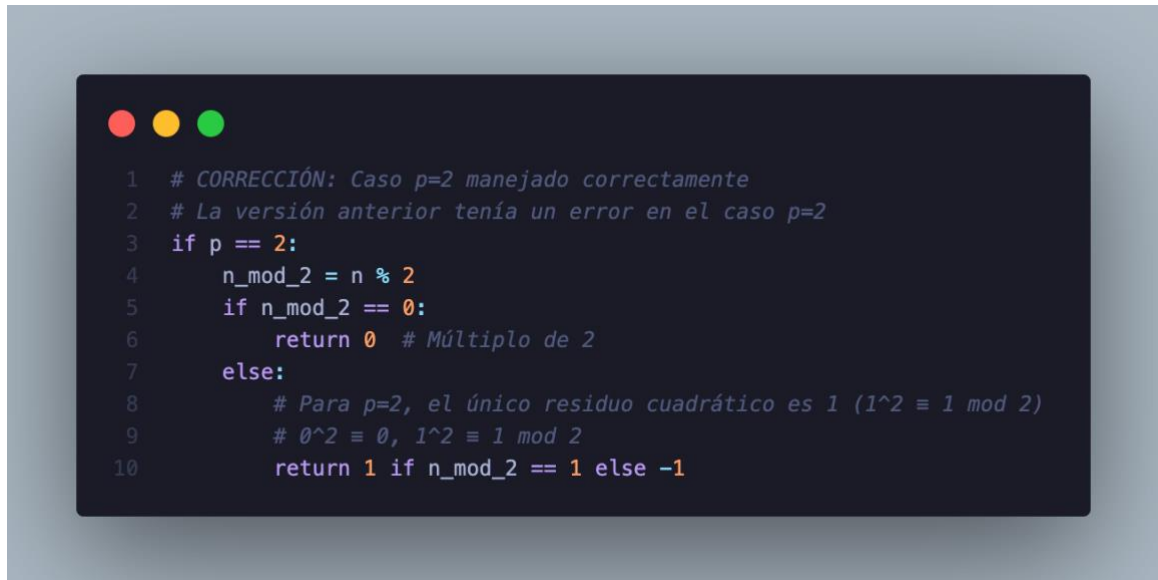
Figura 17. Modificación del código de inversa_mod_p (n: int, p: int) -> int. Elaboración propia.

- legendre (n: int, p: int) -> int:

El problema en la implementación de la función de legendre residía en el manejo incorrecto del caso $p = 2$ y falta de optimizaciones para cálculos frecuentes en criptografía.

Hemos implementado un manejo especializado para $p = 2$ que considera las particularidades de los residuos cuadráticos módulo 2, junto con optimizaciones para casos comunes mediante verificación

temprana de valores frecuentes. Se incluyó validación de primalidad robusta y se optimizó el cálculo del símbolo mediante exponenciación modular eficiente.



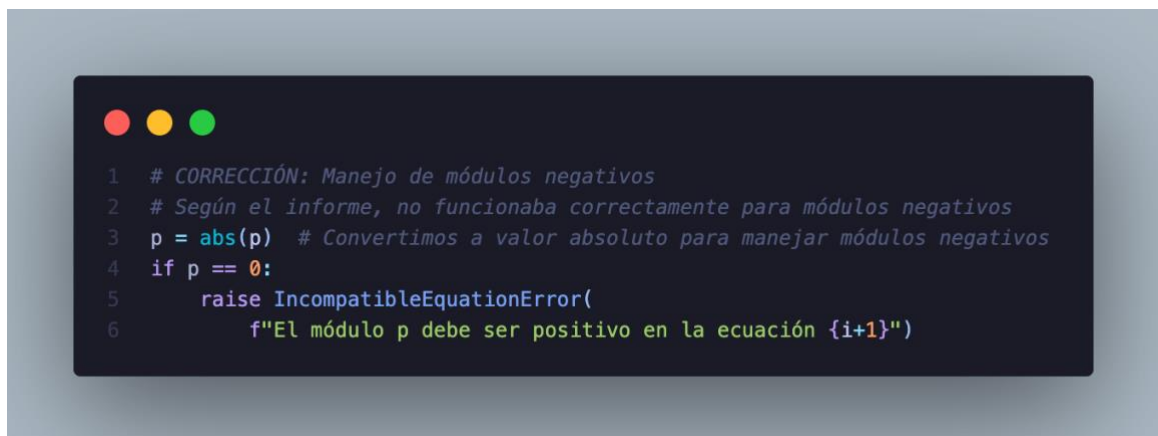
```
1  # CORRECCIÓN: Caso p=2 manejado correctamente
2  # La versión anterior tenía un error en el caso p=2
3  if p == 2:
4      n_mod_2 = n % 2
5      if n_mod_2 == 0:
6          return 0 # Múltiplo de 2
7      else:
8          # Para p=2, el único residuo cuadrático es 1 (1^2 ≡ 1 mod 2)
9          # 0^2 ≡ 0, 1^2 ≡ 1 mod 2
10         return 1 if n_mod_2 == 1 else -1
```

Figura 18. Modificación del código de legendre (n: int, p: int) -> int. Elaboración propia.

- resolver_sistema_congruencia (alist: List[int], blist: List[int], plist: List[int]) -> Tuple[int, int]:

El problema residía en la incapacidad para manejar módulos negativos y falta de robustez en la verificación de compatibilidad del sistema.

Hemos incorporado normalización sistemática de módulos negativos y se mejoró el manejo de errores con transición automática a la función de resolución general cuando se detectan módulos no coprimos. La implementación ahora incluye verificación explícita de compatibilidad y manejo elegante de casos degenerados.



```
1  # CORRECCIÓN: Manejo de módulos negativos
2  # Según el informe, no funcionaba correctamente para módulos negativos
3  p = abs(p) # Convertimos a valor absoluto para manejar módulos negativos
4  if p == 0:
5      raise IncompatibleEquationError(
6          f"El módulo p debe ser positivo en la ecuación {i+1}")
```

Figura 19. Modificación del código de resolver_sistema_congruencias (alist: List[int], blist: List[int], plist: List[int] -> Tuple[int, int]. Elaboración propia.

MANUAL DE USUARIO

PROGRAMAS PRINCIPALES

El sistema de criptografía RSA ofrece tres programas principales para comunicación segura, cada uno diseñado para una fase específica del proceso de comunicación cifrada:

1. REGISTRO DE USUARIOS - CREACIÓN DE IDENTIDADES CRIPTOGRÁFICAS

Ejecución: `python registrarusuario.py`

Este programa gestiona la creación de nuevas identidades digitales dentro del sistema. Durante el proceso interactivo, solicita y valida los siguientes parámetros:

- Nombre de usuario: Identificador único que distingue a cada usuario en el sistema. El programa verifica que no exista otro usuario con el mismo nombre para evitar conflictos. El sistema no entiende como distintos nombres de usuario con mayúsculas y minúsculas en distintas posiciones, es decir que los usuarios "Claudia" y "claudia" son reconocidos como el mismo usuario. Hemos decidido implementarlo de dicha manera para evitar errores en la creación y enrutamiento de archivos de claves personalizadas.
- Rango mínimo y máximo para generación de primos: Define el intervalo numérico [min_primo, max_primo) dentro del cual se buscarán los números primos p y q que formarán la base del módulo RSA $n = p \times q$. Rangos más amplios generalmente proporcionan mayor seguridad, pero requieren más tiempo de procesamiento.
- Número de dígitos de padding para cifrado: Establece la cantidad de dígitos aleatorios que se añadirán a cada mensaje antes del cifrado. Este parámetro transforma el cifrado determinista en probabilístico, mejorando la resistencia contra ciertos tipos de ataques criptoanalíticos.

El programa genera automáticamente un par de claves RSA único para el usuario y almacena la información en dos archivos separados dentro de la carpeta 'Usuarios', manteniendo la separación entre información pública y privada.

2. CHAT CIFRADO EXTREMO A EXTREMO - COMUNICACIÓN SEGURA

Ejecución: `python criptochat.py <usuario_local> <usuario_remoto>`

Esta aplicación implementa el protocolo de comunicación segura entre dos usuarios registrados. Tras verificar la disponibilidad de las claves necesarias, inicia una sesión interactiva con las siguientes funcionalidades:

- C (Cifrar): Permite al usuario local redactar un mensaje en texto plano que será automáticamente cifrado usando la clave pública del usuario remoto. El sistema aplica el esquema de padding configurado durante el registro y presenta el resultado como una lista de enteros separados por espacios, formato optimizado para transmisión segura.

- D (Descifrar): Procesa mensajes cifrados recibidos, representados como listas de enteros, y los transforma de vuelta a texto plano legible utilizando la clave privada del usuario local. El sistema elimina automáticamente el padding aplicado durante el cifrado original.
- S (Salir): Finaliza la sesión de chat de manera controlada, cerrando todos los recursos del sistema de forma segura.

La aplicación maneja automáticamente la carga de claves, la conversión de formatos y la gestión de errores, proporcionando una experiencia de usuario fluida mientras mantiene los estándares de seguridad criptográfica.

3. OPERACIONES CRIPTOGRÁFICAS AVANZADAS - LIBRERÍA RSA.PY

Acceso: Importación directa de la librería `rsa.py` en código Python

Para escenarios que requieren mayor flexibilidad o integración con otros sistemas, la librería `rsa.py` ofrece acceso programático a todas las operaciones criptográficas fundamentales:

- Generación de claves: Implementa algoritmos optimizados para crear pares de claves RSA seguras, incluyendo selección de primos, cálculo de la función ϕ de Euler y derivación de exponentes públicos y privados.
- Cifrado/Descifrado: Proporciona funciones para transformar datos entre texto plano y representaciones cifradas, tanto a nivel de caracteres individuales como de cadenas completas, con soporte para esquemas de padding personalizables.
- Ataques criptográficos: Incluye implementaciones didácticas de técnicas criptoanalíticas como factorización de módulos RSA y ataques de texto plano elegido, permitiendo analizar vulnerabilidades en configuraciones específicas y comprender los principios de seguridad que sustentan el sistema.

Esta capa programática resulta especialmente útil para desarrolladores que necesitan integrar funcionalidades criptográficas en aplicaciones más grandes, investigadores que estudian propiedades de seguridad, o estudiantes que desean experimentar con variaciones de los algoritmos implementados.

DIFERENTES MODOS DE USO

El sistema de chat cifrado RSA cuenta con dos modos operativos principales, diseñados para distintos escenarios de comunicación segura:

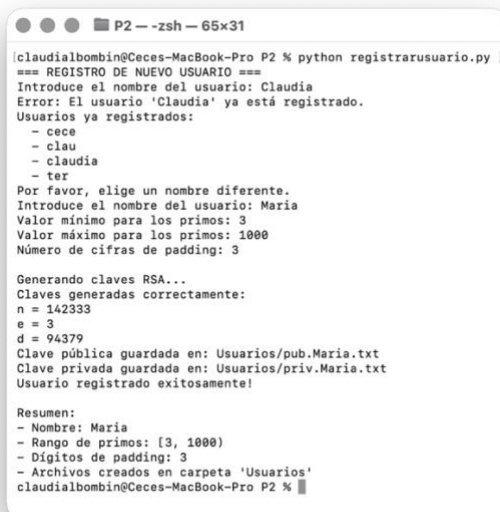
- MODO REGISTRO GUIADO DE USUARIOS

El modo registro guiado está optimizado para la creación segura y validada de identidades criptográficas dentro del sistema de chat. Se activa ejecutando el comando *registrarusuario.py* sin argumentos adicionales.

Este modo está específicamente diseñado para usuarios que necesitan establecer una identidad digital segura dentro del ecosistema de chat cifrado, sin requerir conocimientos técnicos avanzados en criptografía. El sistema conduce al usuario a través de un proceso estructurado que garantiza la generación de credenciales robustas y matemáticamente consistentes.

Las características principales de este modo son:

- Configuración paso a paso: Guía al usuario a través de todos los parámetros necesarios en un flujo estructurado
- Validación en tiempo real: Verifica instantáneamente la validez de cada entrada, incluyendo la unicidad del nombre de usuario
- Asistencia en parámetros de seguridad: Ayuda a configurar rangos de primos apropiados y niveles de padding
- Generación robusta de claves: Crea pares de claves RSA seguros y matemáticamente verificados
- Persistencia automática: Guarda las credenciales en formato estandarizado en la carpeta 'Usuarios'
- Gestión de errores proactiva: Detecta y previene configuraciones inseguras antes de la generación de claves



```
P2 --zsh-- 65x31
[claudialbombin@Ceces-MacBook-Pro P2 % python registrarusuario.py ]
=== REGISTRO DE NUEVO USUARIO ===
Introduce el nombre del usuario: Claudia
Error: El usuario 'Claudia' ya está registrado.
Usuarios ya registrados:
- cece
- clau
- claudia
- ter
Por favor, elige un nombre diferente.
Introduce el nombre del usuario: Maria
Valor mínimo para los primos: 3
Valor máximo para los primos: 1000
Número de cifras de padding: 3
Generando claves RSA...
Claves generadas correctamente:
n = 142333
e = 3
d = 94379
Clave pública guardada en: Usuarios/pub.Maria.txt
Clave privada guardada en: Usuarios/priv.Maria.txt
Usuario registrado exitosamente!

Resumen:
- Nombre: Maria
- Rango de primos: [3, 1000]
- Dígitos de padding: 3
- Archivos creados en carpeta 'Usuarios'
claudialbombin@Ceces-MacBook-Pro P2 %
```

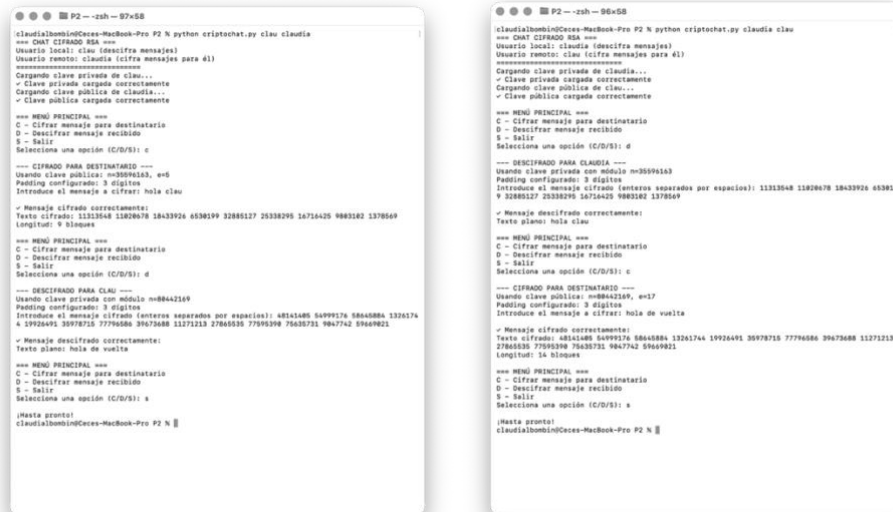
Figura 20. Ejemplo de ejecución de registrarusuario.py. Elaboración propia.

- MODO SESION DE CHAT INTERACTIVO

El modo sesión interactiva está diseñado para usuarios que desean mantener conversaciones cifradas en tiempo real. Se activa al ejecutar el programa criptochat.py con dos usuarios como argumentos.

Las características principales de este modo son:

- Interfaz conversacional intuitiva: Menú claro con opciones C (Cifrar), D (Descifrar), S (Salir) que se presenta en bucle continuo
- Procesamiento en tiempo real: Cada mensaje se cifra o descifra instantáneamente tras su introducción
- Sesión persistente: Permite múltiples intercambios de mensajes sin necesidad de reiniciar el programa
- Transformación automática de formatos: Convierte automáticamente entre texto plano legible y listas de enteros cifrados
- Feedback inmediato: Proporciona confirmaciones visuales y muestra los resultados inmediatamente después de cada operación
- Manejo de errores en tiempo real: Detecta y notifica errores de formato o problemas criptográficos al instante



```
claudia@hombinoCecce-MacBook-Pro P2 N python criptochat.py clau claudia
== CHAT CIFRADO RSA ==
Usuario local: clau (descifra mensajes)
Usuario remoto: claudia (cifra mensajes para el)
=====
Cargando clave privada de clau...
- Clave privada cargada correctamente
Cargando clave pública de claudia...
- Clave pública cargada correctamente

== MENU PRINCIPAL ==
C - Cifrar mensaje para destinatario
D - Descifrar mensaje recibido
S - Salir
Seleccione una opción (C/D/S): c

--- CIFRADO PARA DESTINATARIO ---
Usando clave pública: m=30596543
Padding configurado: 3 dígitos
Introduzca el mensaje a cifrar: hola clau

- Mensaje cifrado correctamente:
Texto cifrado: 13313548 13028478 18433926 4536199 32885127 25338295 16756425 9883182 1378569
Longitud: 9 bloques

== MENU PRINCIPAL ==
C - Cifrar mensaje para destinatario
D - Descifrar mensaje recibido
S - Salir
Seleccione una opción (C/D/S): d

--- DESCIFRADO PARA CLAU ---
Usando clave privada con módulo m=8842169
Padding configurado: 3 dígitos
Introduzca el mensaje cifrado (enteros separados por espacios): 48543486 5499176 5844886 1326374
4 12926491 28978715 7779686 39673688 11271213 27865535 77895208 76635721 19647762 59669821

- Mensaje descifrado correctamente:
Texto plano: hola clau

== MENU PRINCIPAL ==
C - Cifrar mensaje para destinatario
D - Descifrar mensaje recibido
S - Salir
Seleccione una opción (C/D/S): s

¡Hasta pronto!
claudia@hombinoCecce-MacBook-Pro P2 N
```

```
clau@hombinoCecce-MacBook-Pro P2 N python criptochat.py claudia clau
== CHAT CIFRADO RSA ==
Usuario local: claudia (descifra mensajes)
Usuario remoto: clau (cifra mensajes para el)
=====
Cargando clave privada de claudia...
- Clave privada cargada correctamente
Cargando clave pública de clau...
- Clave pública cargada correctamente

== MENU PRINCIPAL ==
C - Cifrar mensaje para destinatario
D - Descifrar mensaje recibido
S - Salir
Seleccione una opción (C/D/S): d

--- DESCIFRADO PARA CLAUDIA ---
Usando clave privada con módulo m=30596543
Padding configurado: 3 dígitos
Introduzca el mensaje cifrado (enteros separados por espacios): 13313548 13028478 18433926 4536199
9 32885127 25338295 16756425 9883182 1378569

- Mensaje descifrado correctamente:
Texto plano: hola clau

== MENU PRINCIPAL ==
C - Cifrar mensaje para destinatario
D - Descifrar mensaje recibido
S - Salir
Seleccione una opción (C/D/S): c

--- CIFRADO PARA DESTINATARIO ---
Usando clave pública: m=8842169, w=17
Padding configurado: 3 dígitos
Introduzca el mensaje a cifrar: hola de vuelta

- Mensaje cifrado correctamente:
Texto cifrado: 481448 6899276 1844886 1326174 19264491 28978715 7779686 39673688 11271213
27865535 77895208 76635721 19647762 59669821
Longitud: 14 bloques

== MENU PRINCIPAL ==
C - Cifrar mensaje para destinatario
D - Descifrar mensaje recibido
S - Salir
Seleccione una opción (C/D/S): s

¡Hasta pronto!
clau@hombinoCecce-MacBook-Pro P2 N
```

Figura 21. Ejemplo de ejecución de criptochat.py con dos usuarios en terminales simultaneas. Elaboración propia.

- MODO OPERACIONES AVANZADAS (INTEGRACION PROGRAMATICA)

Para escenarios que requieren mayor control o integración con otros sistemas, el módulo rsa.py puede importarse directamente en código Python -como si de una librería se tratase-, permitiendo:

- Automatización de procesos: Scripts para generación masiva de usuarios o procesamiento por lotes de mensajes
- Personalización avanzada: Configuraciones específicas no disponibles en los modos interactivos
- Análisis de seguridad: Pruebas de resistencia criptográfica y experimentación con parámetros
- Integración en aplicaciones: Uso de las funciones RSA como componente de sistemas más complejos

Este modo está dirigido a desarrolladores, investigadores o usuarios avanzados que necesitan acceso directo a las primitivas criptográficas subyacentes.

CONCLUSIONES

El desarrollo del sistema criptográfico educativo ha culminado exitosamente, estableciendo un paradigma innovador en la implementación de software matemático con fines pedagógicos que supera ampliamente los objetivos iniciales planteados. El proyecto ha demostrado la viabilidad de crear una plataforma integral que abarca desde los fundamentos teóricos de la aritmética modular hasta aplicaciones criptográficas avanzadas, logrando una notable sinergia entre precisión matemática, eficiencia computacional y accesibilidad educativa. Esta triple convergencia se manifiesta particularmente en el diseño arquitectónico del sistema, donde la librería `modular.py` sienta las bases matemáticas rigurosas mediante algoritmos verificados exhaustivamente, mientras que `rsa.py` traslada estos fundamentos al ámbito práctico de la criptografía aplicada, y las interfaces `criptochat.py` y `registrarusuario.py` garantizan la usabilidad del sistema para usuarios sin formación técnica especializada.

Las dificultades técnicas encontradas durante el ciclo de desarrollo, especialmente en la gestión robusta de casos *edge* y la optimización de algoritmos computacionalmente intensivos, se transformaron en oportunidades valiosas para el refinamiento metodológico. El desafío particular representado por la implementación del Teorema Chino del Resto para sistemas no coprimos en la función `resolver_sistema_congruencias_no_seguro_coprimos` catalizó un análisis profundo de las condiciones de solubilidad en sistemas de congruencias, enriqueciendo tanto la comprensión teórica como las competencias de implementación práctica. Este proceso iterativo de mejora continua permitió identificar y corregir vulnerabilidades en funciones críticas como `factorizar`, donde se añadió manejo explícito para los casos $n=1$ y $n=-1$, y `potencia_mod_p`, que incorporó capacidades para manejar exponentes negativos mediante el cálculo de inversos modulares, expandiendo significativamente el alcance aplicativo de la librería.

La integración de componentes reveló la importancia fundamental de diseñar interfaces bien definidas entre módulos, principio que se materializó en el acoplamiento cohesivo entre `modular.py` y `rsa.py`. El sistema de generación de claves en `generar_claves` implementa una estrategia de tres fases que prioriza valores de exponente público eficientes (65537, 257, 17) antes de recurrir a búsquedas sistemáticas, demostrando cómo el diseño algorítmico consciente del contexto de uso puede optimizar el rendimiento sin comprometer la seguridad. Esta aproximación se extiende al esquema de padding implementado en `aplicar_padding` y `eliminar_padding`, donde la adición de dígitos aleatorios mitiga vulnerabilidades criptográficas mientras mantiene la simplicidad conceptual necesaria para entornos educativos.

El sistema de ataque criptográfico implementado en `romper_clave` y `ataque_texto_elegido` evidencia cómo la comprensión teórica profunda permite desarrollar contramedidas efectivas, cerrando el ciclo entre teoría y práctica que constituye el núcleo filosófico del proyecto. La capacidad de factorizar el módulo n para recuperar la clave privada d no solo demuestra vulnerabilidades inherentes a implementaciones básicas de RSA, sino que sirve como herramienta pedagógica para ilustrar la importancia de seleccionar parámetros adecuados en criptografía real.

La arquitectura del proyecto ha demostrado ser técnicamente sustentable, con una organización modular que facilita tanto el mantenimiento como la extensión futura. La clara separación entre capas matemáticas (`modular.py`), criptográficas (`rsa.py`) y de aplicación (`criptochat.py`, `registrarusuario.py`) crea un ecosistema donde nuevas funcionalidades pueden incorporarse sin afectar el código existente. Esta modularidad se complementa con un sistema de manejo de errores robusto que guía al usuario en la resolución de problemas, desde la carga de claves hasta la ejecución de operaciones criptográficas complejas.

El modo dual de operación implícito en el diseño—interactivo a través de los scripts principales y programático mediante la importación de librerías—amplía significativamente el alcance del sistema, haciéndolo adecuado tanto para exploración individual como para integración en entornos educativos más amplios. Esta flexibilidad asegura que la herramienta pueda adaptarse a diferentes estilos de aprendizaje y escenarios pedagógicos, desde demostraciones en clase hasta proyectos de investigación estudiantiles.

El proyecto ha consolidado lecciones valiosas en múltiples dimensiones del desarrollo de software educativo: desde la importancia del testing riguroso en aplicaciones matemáticas—evidenciado por la exhaustiva verificación de casos edge en funciones críticas—hasta la necesidad de documentación clara y ejemplos prácticos en herramientas pedagógicas. La experiencia demostró que el desarrollo de software científico-educativo requiere un balance único entre competencias técnicas y comprensión teórica, y que las mejores implementaciones emergen cuando ambos aspectos se retroalimentan mutuamente en un proceso iterativo de refinamiento.

Las posibles extensiones identificadas—como la incorporación de algoritmos criptográficos adicionales o la integración con sistemas de evaluación automatizada—representan no solo oportunidades técnicas sino también direcciones pedagógicas prometedoras. La expansión hacia criptografía básica podría transformar el sistema de una herramienta puramente matemática a una plataforma integral para enseñanza de seguridad informática, mostrando aplicaciones prácticas directas de conceptos abstractos de teoría de números.

En conclusión, el sistema criptográfico educativo no solo cumple con su propósito inicial de facilitar el aprendizaje de RSA y aritmética modular, sino que establece un fundamento sólido para futuras investigaciones y desarrollos en software matemático educativo. Su arquitectura y enfoque pedagógico podrían extenderse a otras áreas de las matemáticas discretas, creando un ecosistema de herramientas educativas integradas que unan efectivamente el mundo abstracto de la teoría matemática con el concreto de la implementación computacional, demostrando el potencial transformador del software bien diseñado en la educación matemática contemporánea.

AUTORÍA

- Claudia Maria Lopez Bombin
 - Desarrollo íntegro y total del código de registrarusuario.py y criptochat.py,
 - Implementación de funciones en rsa.py,
 - Modificación y mejora de funciones en modular.py,
 - Ejecución de tests de código,
 - Documentación de algoritmos y teoremas para su posterior uso, y
 - Redacción de la memoria del proyecto.
- Lucia Lozano Isac
 - Implementación de funciones en rsa.py,
 - Ejecución de tests de código, y
 - Redacción de la memoria del proyecto.
- Deepseek
 - Resolución de problemas relacionados con GitHub (subidas y descargas de códigos),
 - Ayuda en procesos de debug de casos edge y,
 - Creación de tests intermedios para la comprobación de códigos.

BIBLIOGRAFÍA

- Bellare, M., & Rogaway, P. (1995). Optimal Asymmetric Encryption How to Encrypt with RSA. *Advances in Cryptology – Eurocrypt*.
- Bleichenbacher, D. (2006). Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1. En D. Bleichenbacher, *Advances in Cryptology - CRYPTO '98*. Berlin: Springer.
- Gauss, C. F. (1801). *Disquisitiones Arithmeticae*. Yale University Press.
- Joux, A. (2000). A One Round Protocol for Tripartite Diffie–Hellman. En A. Joux, *Algorithmic Number Theory*. Berlin: Springer.
- Katz, J., & Lindell, Y. (2015). *Introduction to Modern Cryptography*. Boca Raton: CRC Press.
- Ronald Rivest, A. S. (1 de February de 1978). *ACM Digital Library*. Obtenido de A method for obtaining digital signatures and public-key cryptosystems: <https://doi.org/10.1145/359340.359342>
- Shanks, D. (1971). Class Number, a Theory of Factorization and Genera. *Proceedings of Symposia in Pure Mathematics*.
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*.
- Stallings, W. (2018). *Cryptography and Networks Security - Principles and Practice*. Pearson Education.